bit.ly/icad25-workshop

Interpretable Model Learning for Taskable AI Systems

MIT CSAIL

Pulkit Verma
pulkitv@mit.edu
pulkitverma.net

Automated Planning

- To complete any task, we plan using a sequence of actions.

- *Automated Planning*:
  - Has discrete high-level actions.
  - Each high-level action corresponds to a sequence of low-level actions or controls.
  - Given an initial state and a goal, what sequence of actions, called a plan, should an AI system execute to achieve the goal?

Interpretable Description: PDDL/PPDDL

```
(:action open-door
  :parameters (?l1)
  :precondition (and
     (has_key)
     (player_at ?l1)
     (door_adjacent ?l1))
  :effect (probabilistic
     0.95 (and (door_open))
     0.05 (and (not (has_key))
               (game-over))
)
```

Precondition: This condition must be true for this action to execute

Effect: This is a set of conditions, one of which becomes true when this action is executed

Probabilities: Each set of effect has an associated probability with which that effect set is executed

Interpretable: Easily Convertible to Natural Language

```
(:action open-door
  :parameters (?l1)
  :precondition (and
     (has_key)
     (player_at ?l1)
     (door_adjacent ?l1))
  :effect (probabilistic
     0.95 (and (door_open))
     0.05 (and (not(has_key))
               (game-over))
)
```

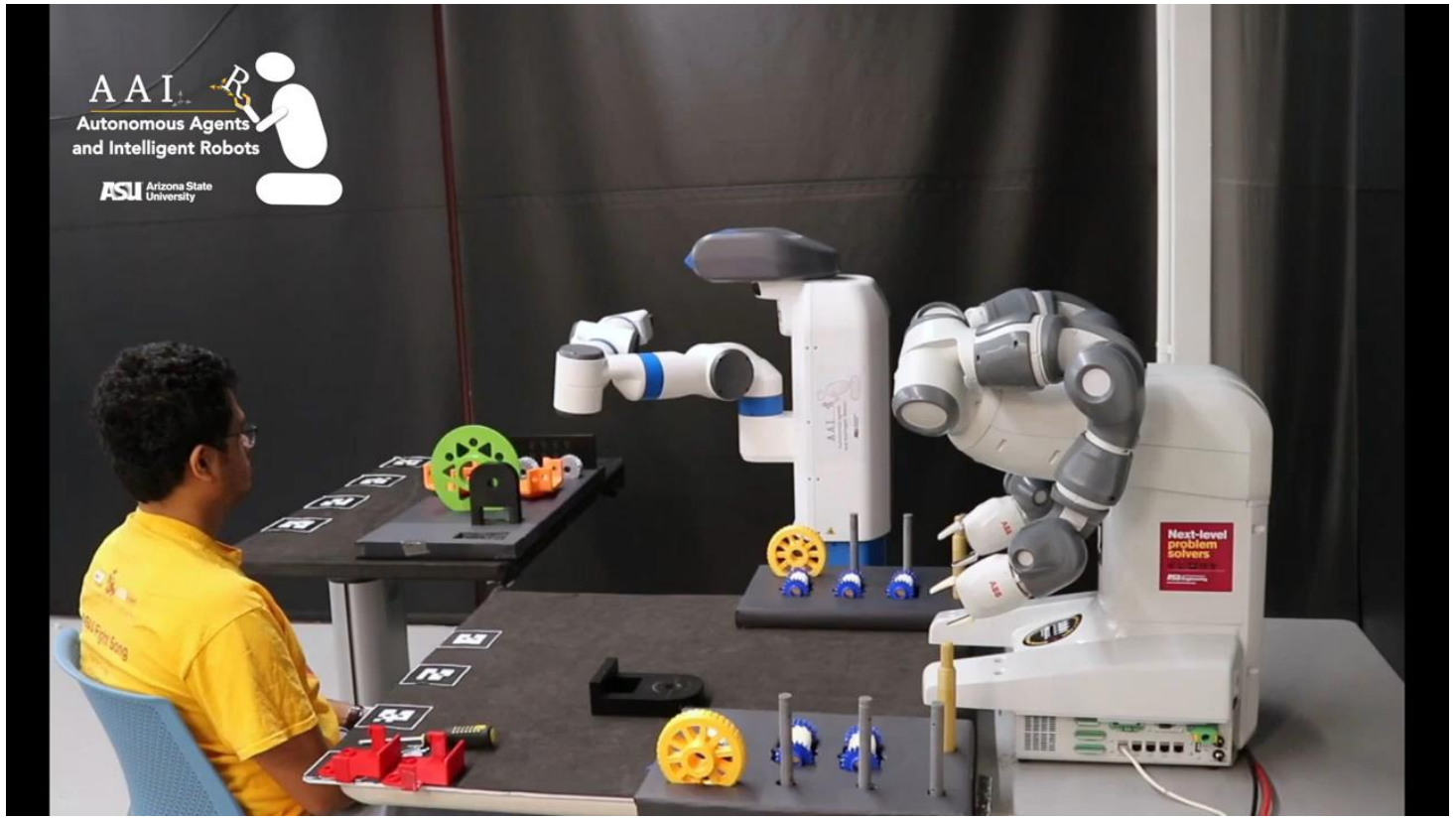
The player can open the door when in location ?l1 if:

- It has the key
- The player is at location ?l1
- The door is adjacent to location ?l1

After executing that capability:

- With 95% probability, the door will open
- With 5% probability, the player will not have the key and the game will be over

Taskable AI Systems: Systems that can Learn and Plan



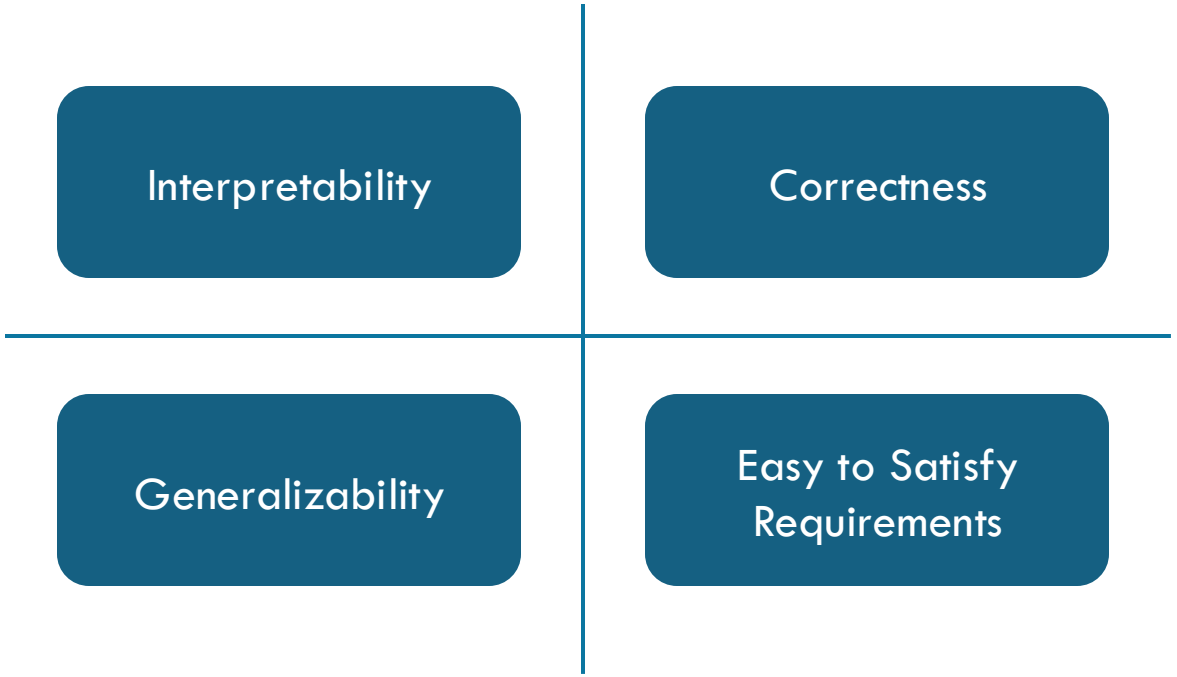
User gives a task and Robots have to complete it

- Sequential Decision-Making Systems.
- Systems designed to be able to help user.
- User has a task in mind and expects the AI system to help in that task.

Personalized Assessment of AI Systems that can Learn and Plan

- Users would like to give AI systems multiple tasks.
  - How would users know what the AI systems can do?
- AI systems should support third-party assessment.
- The assessment should work with:
  - Adaptive AI systems.
  - Black-Box AI systems.

Desiderata of Assessment System



The Assessment Problem

Will it be able to safely rearrange my lab for the next round of experiments?

Output

- A description of agent's working:
  - A list of capabilities.
  - An interpretable description of each capability.

Black-Box AI

- Arbitrary internal implementation
- Comes with a Trained Policy

Capability v/s Functionality

- ***Capability***
  Robot can move an item from anywhere in the room to the shelf.

- ***Functionality***
  Robot can move one step to its left, right, front, and back.

```
at(p0,cell_6_3)
clear(cell_0_0)
door_at(cell_9_2)
next_to(m0)
alive(m0)
key_at(9_4)
```

[Input]
Concepts that the user understands

Personalized AI-Assessment Module (AAM)

Black-Box AI

Arbitrary internal implementation

Doesn't know user's vocabulary

```
at(p0,cell_6_3)
clear(cell_0_0)
door_at(cell_9_2)
next_to(m0)
alive(m0)
key_at(9_4)
```

[Input]
Concepts that the user understands

Personalized AI-Assessment Module (AAM)

Query

Response

simulator

Arbitrary internal implementation

Doesn't know user's vocabulary

Black-Box AI

[Input]
Concepts that the user understands

[Output]
User-Interpretable description of Black-Box AI's capabilities

Personalized AI-Assessment Module (AAM)

This Talk

Query

Response

simulator

Black-Box AI

Arbitrary internal implementation

Doesn't know user's vocabulary

The Assessment Problem

Black-Box (Taskable) AI

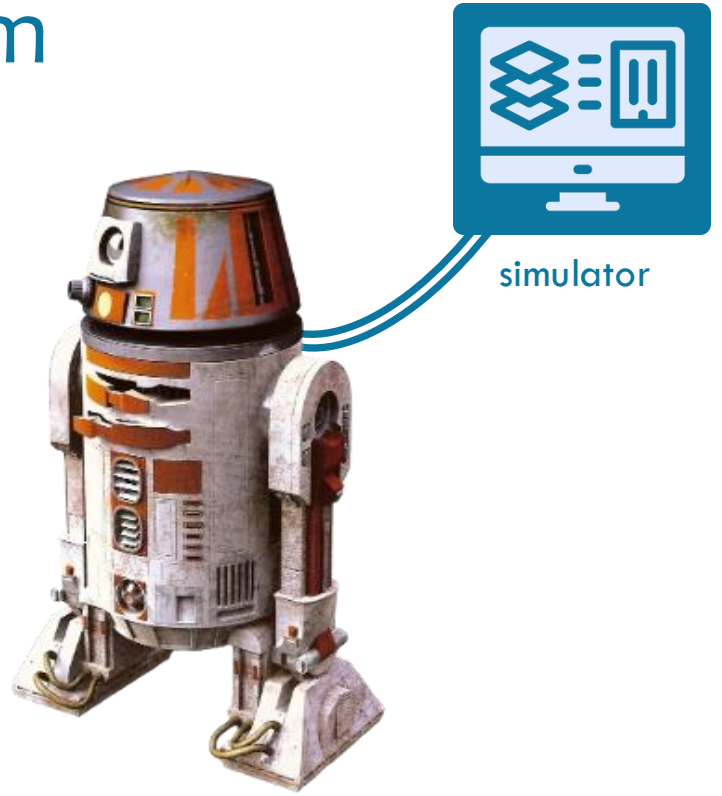
- Can be connected to a simulator.
- Can have arbitrary internal implementation.
- Does not know input vocabulary.

Input

- Predicates (User vocabulary)
  - With their evaluation functions

Output

- A description of agent's working:
  - A list of capabilities.
  - An interpretable description of each capability.



Black-Box AI

Assessment using Passive Observations

$$\langle s_0, a_1, s_1 \rangle$$
$$\langle s_1, a_2, s_2 \rangle$$
$$\vdots$$
$$\langle s_{n-1}, a_n, s_n \rangle$$

[Input]

Learner

```
(:action pick-up-beaker
   :parameters (?x - beaker)
      :precondition (and (ontable ?x)
         (not-in-use ?x)
         (handempty))
      :effect (and (not (ontable ?x))
          (not (handempty))
          (holding ?x)))

(:action put-on-shelf
   :parameters (?x - beaker)
      :precondition (holding ?x)
      :effect (and (not (holding ?x))
          (handempty)
          (onshelf ?x)))
```

[PDDL Example]

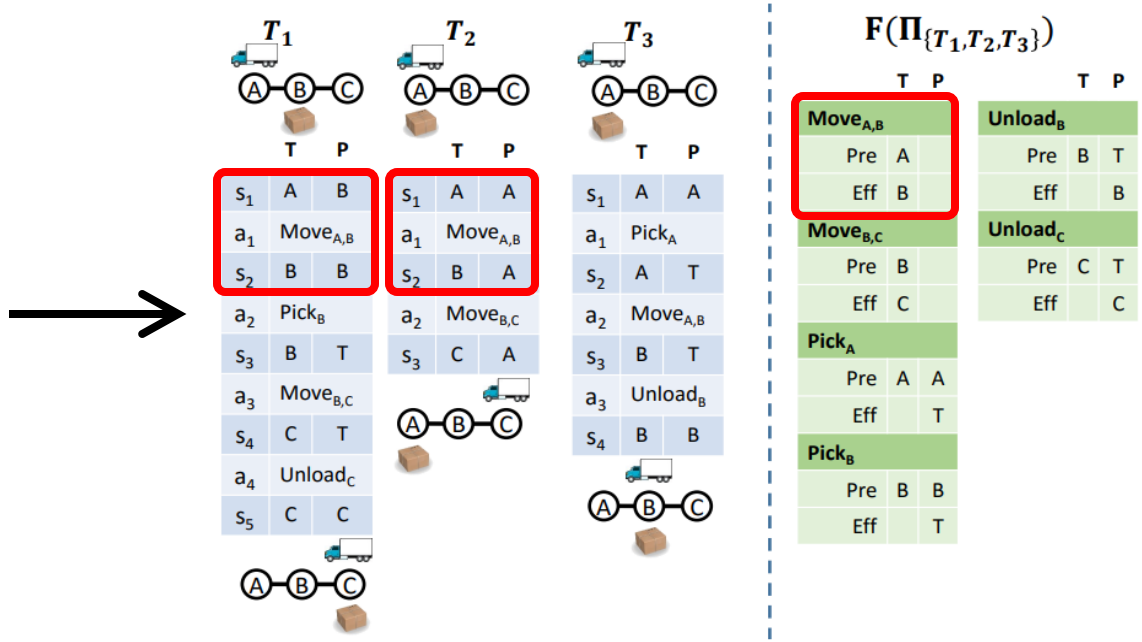
What kind of approaches these learners use?

Inference Rules based Learners

- Take intersection of all states where an action is applicable to create precondition.
- Take intersection of all states after executing an action to create effect.

$\langle s_0, a_1, s_1 \rangle$
$\langle s_1, a_2, s_2 \rangle$
$\vdots$
$\langle s_{n-1}, a_n, s_n \rangle$

[Input]




```
(:action move-A-B
  :parameters ()
  :precondition (and (at A))
  :effect (and (not (at A)) (at B))
)

(:action move-B-C
  :parameters ()
  :precondition (and (at B))
  :effect (and (not (at B)) (at C))
)
```

[PDDL Example]

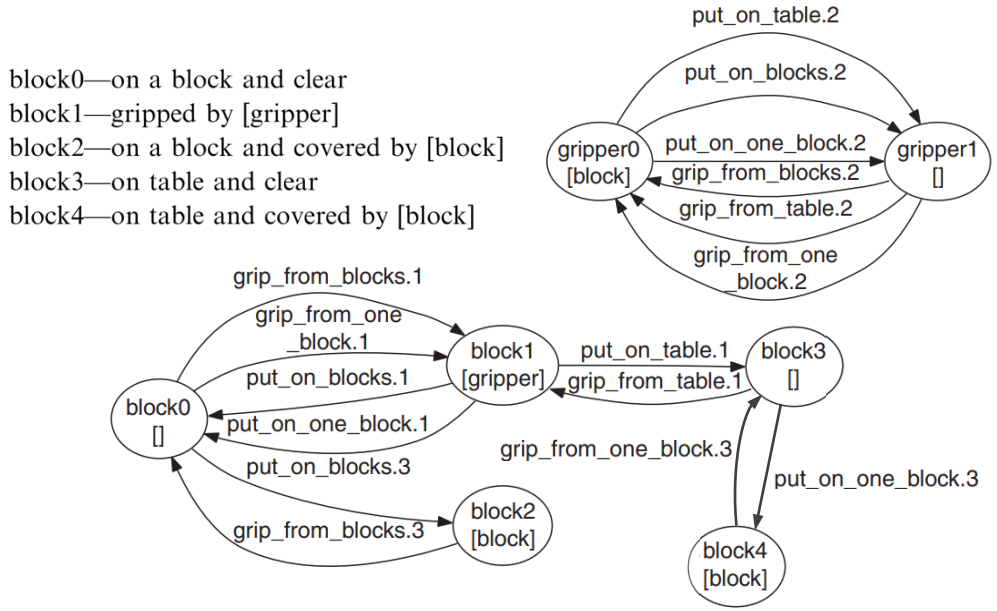
Stern et al. (IJCAI'17), SLAM - Juba et al. (KR'21)

Finite State Machine based Learners

- For each object type create a finite state machine.
- Create PDDL by combining them.

$$\langle s_0, a_1, s_1 \rangle$$
$$\langle s_1, a_2, s_2 \rangle$$
$$\vdots$$
$$\langle s_{n-1}, a_n, s_n \rangle$$

[Input]




```
(:action pick-up-beaker
   :parameters (?x - beaker)
      :precondition (and (ontable ?x)
         (not-in-use ?x)
         (handempty))
      :effect (and (not (ontable ?x))
          (not (handempty))
          (holding ?x)))

(:action put-on-shelf
   :parameters (?x - beaker)
      :precondition (holding ?x)
      :effect (and (not (holding ?x))
          (handempty)
          (onshelf ?x)))
```

[PDDL Example]

LOCM - Cresswell et al. (ICAPS'09), LOCM2 - Cresswell et al. (ICAPS'11, Know. Engg. Rev.'13),
LOP - Gregory et al. (ICAPS'15), NLOCM - Gregory et al. (ICAPS'16)

SAT based Learners

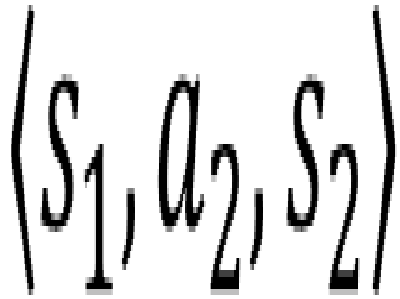
- Create a SAT problem using constraint axioms.
- Extract PDDL from the SAT problem's solution.

$\langle s_0, a_1, s_1 \rangle$
$\langle s_1, a_2, s_2 \rangle$
$\vdots$
$\langle s_{n-1}, a_n, s_n \rangle$

[Input]

1. $(par(p_k) \cap par(pri_i) = \phi) \wedge (par(p_k) \cap par(pre_{goal}) = \phi) \Rightarrow p_k \notin add_i \wedge p_k \notin del_i$
2. $pre_i \neq \phi \wedge add_i \neq \phi \wedge del_i \neq \phi$
3. $pre_i \cap add_i = \phi$
4. $del_i \subseteq pre_i$.

.
.
.



$\langle s_1, a_2, s_2 \rangle$

[PDDL Example]

ARMS -Yang et al. (AIJ 2007), Zhuo et al. (IJCAI'13)

Planning based Learners

- Create Planning problem using SAT-like rules.
- Extract correct PDDL from solution to the planning problem.

$$\langle s_0, a_1, s_1 \rangle$$
$$\langle s_1, a_2, s_2 \rangle$$
$$\vdots$$
$$\langle s_{n-1}, a_n, s_n \rangle$$

[Input]

```
(:action apply_stack
  :parameters (?o1 - object ?o2 - object)
  :precondition
   (and (or (not (pre_stack_on_v1_v1)) (on ?o1 ?o1))
        (or (not (pre_stack_on_v1_v2)) (on ?o1 ?o2))
        (or (not (pre_stack_on_v2_v1)) (on ?o2 ?o1))
        (or (not (pre_stack_on_v2_v2)) (on ?o2 ?o2))
                    . . .
        (or (not (pre_stack_handempty)) (handempty)))
  :effect
   (and (when (del_stack_on_v1_v1) (not (on ?o1 ?o1)))
        (when (del_stack_on_v1_v2) (not (on ?o1 ?o2)))
        (when (del_stack_on_v2_v1) (not (on ?o2 ?o1)))
        (when (del_stack_on_v2_v2) (not (on ?o2 ?o2)))
                    . . .
        (when (add_stack_holding_v1) (holding ?o1))
        (when (add_stack_holding_v2) (holding ?o2))
        (when (add_stack_handempty) (handempty))
        (when (modeProg) (not (modeProg)))))
```



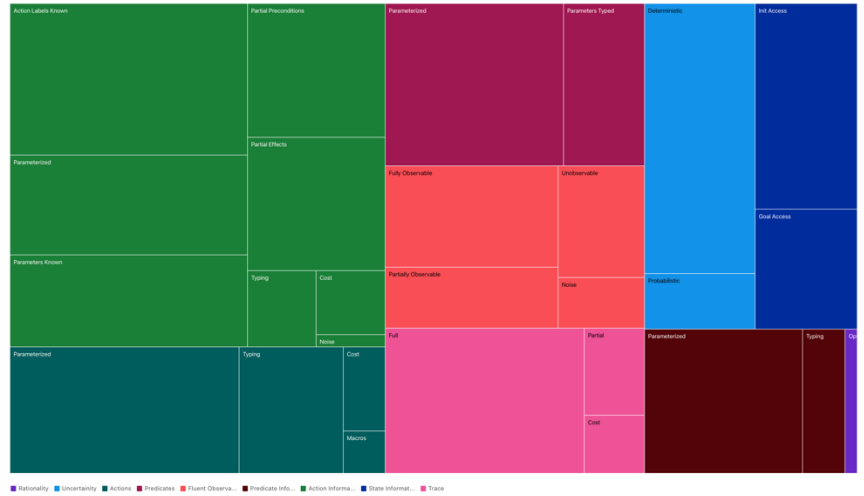
```
(:action pick-up-beaker
   :parameters (?x - beaker)
      :precondition (and (ontable ?x)
         (not-in-use ?x)
         (handempty))
      :effect (and (not (ontable ?x))
          (not (handempty))
          (holding ?x)))

(:action put-on-shelf
   :parameters (?x - beaker)
      :precondition (holding ?x)
      :effect (and (not (holding ?x))
          (handempty)
          (onshelf ?x)))
```

[PDDL Example]

FAMA – Aineto et al. (ICAPS'18, AIJ'19)

MACQ: Model Acquisition Toolkit

- Library of passive learning approaches
- Re-implementations of landm approaches
- Open source
- Visualization tools



Learning Partially Observable Deterministic Action Models. Amir, Eyal, and Allen Chang. JAIR. (2008)

Neighboring Papers

To get to this new paper, our AI thinks you should be looking at the following papers known to our system as the state of the art that immediately makes the new work possible. Each paper is tagged with features that need relaxation or extension to get to the new paper.

Efficient, Safe, and Probably Approximately Complete Learning of Action Models *by Stern, Roni, and Brendan Juba.* IJCAI (2017)	Learning Parameters / Data Features / Trace / Cost / True
Constructing Symbolic Representations for High-Level Planning *by Konidaris, George, Leslie Kaelbling, and Tomas Lozano-Perez.* AAAI (2014)	Learning Parameters / Data Features / State Information / Init Access / False Learning Parameters / Data Features / Trace / Cost / True
Learning First-Order Representations for Planning from Black-Box States: New Results *by Rodriguez, Ivan D., Blai Bonet, Javier Romero, and Hector Geffner.* arXiv (2021)	Learning Parameters / Model Features / Actions / Parameterized / False Learning Parameters / Model Features / Predicates / Parameterized / False Learning Parameters / Data Features / Fluent Observability / Fully Observable / True Learning Parameters / Data Features / Fluent Observability / Unobservable / False Learning Parameters / Data Features / Fluent Observability / Noise / False Learning Parameters / Data Features / Trace / Cost / True

Tutorial on Model Acquisition using MACQ

https://icaps23.icaps-conference.org/program/tutorials/model/

Model Acquisition in the Modern Era (Tutorial Materials)

Description

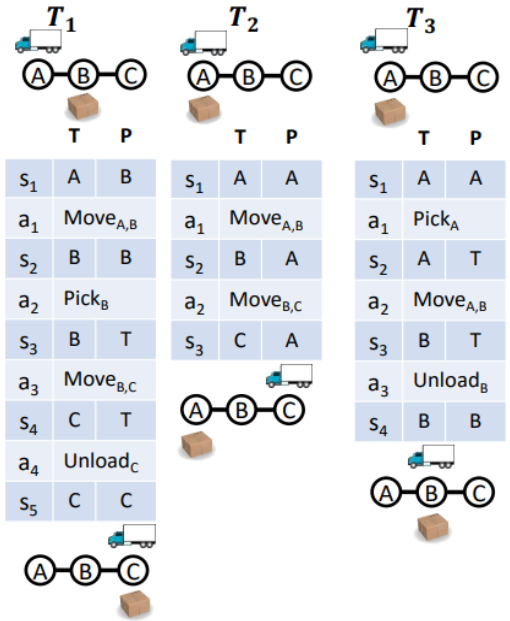
This tutorial will cover some of the landmark methods in the area of planning action model acquisition that our community has produced over the years. From OBSERVER in the early 90's to the modern forms of action-label-only LOCM techniques, we will cover both the concepts behind these approaches and grounded hands-on examples for attendees to try for themselves.

Limitations of Learning from Passive Observations

- Susceptible to incorrect or incomplete model learning.

- E.g., if all packages are brown in color, a possible precondition will be that the package must be brown to unload them.

- Such methods don't capture correct causal relationships.



Active Acquisition of Observations

- Does not depend on third-party to provide observations.
- Strategy to acquire observations:
  - Directed Search: What action should I execute more to acquire more samples?

EXPO – Gil (ICML'93), IRALe – Rodrigues et al. (ILP'11), GLIB – Chitnis et al. (AAAI'21)

Asking the Right Questions: Learning Interpretable Action Models Through Query Answering

Pulkit Verma, Shashank Rao Marpally, and Siddharth Srivastava

AAAI 2021

Deterministic and Stationary Setting

Input

- Predicates (User vocabulary)
  - With their evaluation functions
- List of capabilities.

Output

- PDDL-like description of each capability.

Siddharth Srivastava

Shashank R Marpally

Asking the Right Questions: Learning Interpretable Action Models Through Query Answering

Pulkit Verma, Shashank Rao Marpally, Siddharth Srivastava

In Proceedings of the Thirty-Fifth AAAI Conference on Artificial Intelligence, 2021 (CORE Ranking: A*)

Assumptions

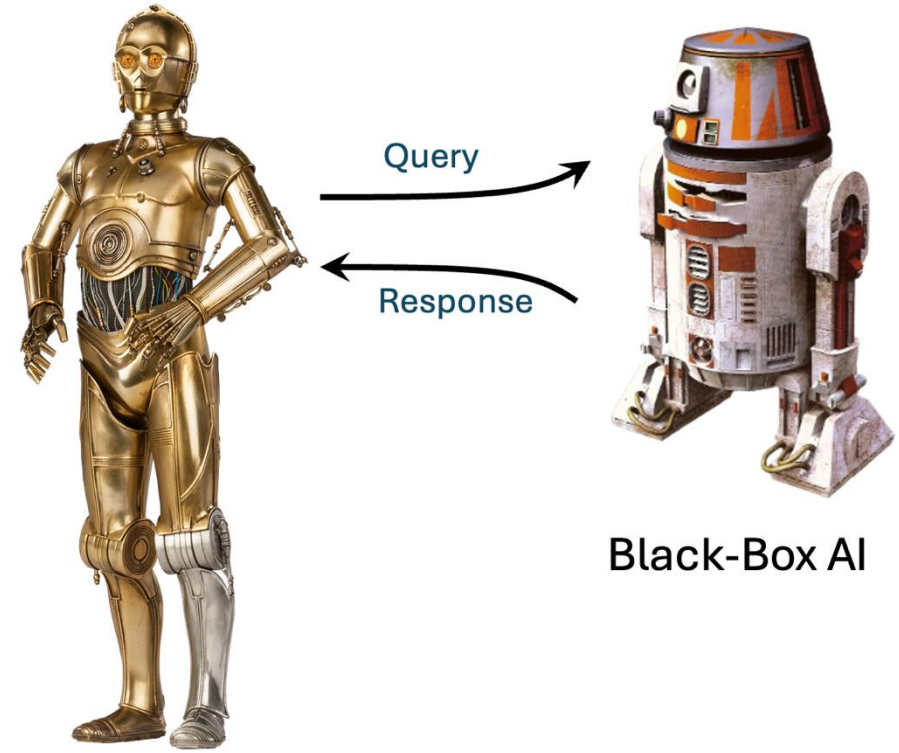
- User's vocabulary matches simulator's vocabulary.
- Black-Box AI provides a list of capabilities.
- Stationary agent model.
- Deterministic environment.
- Fully observable setting.

Exponential Search for Learning Correct Description

- Consider the following 4 predicates/concepts:
  - (has_key)
  - (door_open)
  - (door_adjacent ?x)
  - (player_at ?x)
- Consider just one capability: (open-door ?x)
- $9^{|C|\times|P|} = 9^{1\times4}$=6561 possible models (Assuming deterministic models/ descriptions, i.e., no probabilities).

```
(:action open-door
  :parameters (?l1)
  :precondition (and
    (+/-/∅)(has_key)
    (+/-/∅)(door_open)
    (+/-/∅)(door_adjacent ?l1)
    (+/-/∅)(player_at ?l1))
  :effect (and
    (+/-/∅)(has_key)
    (+/-/∅)(door_open)
    (+/-/∅)(door_adjacent ?l1)
    (+/-/∅)(player_at ?l1))
```

Black-Box AI System Interface



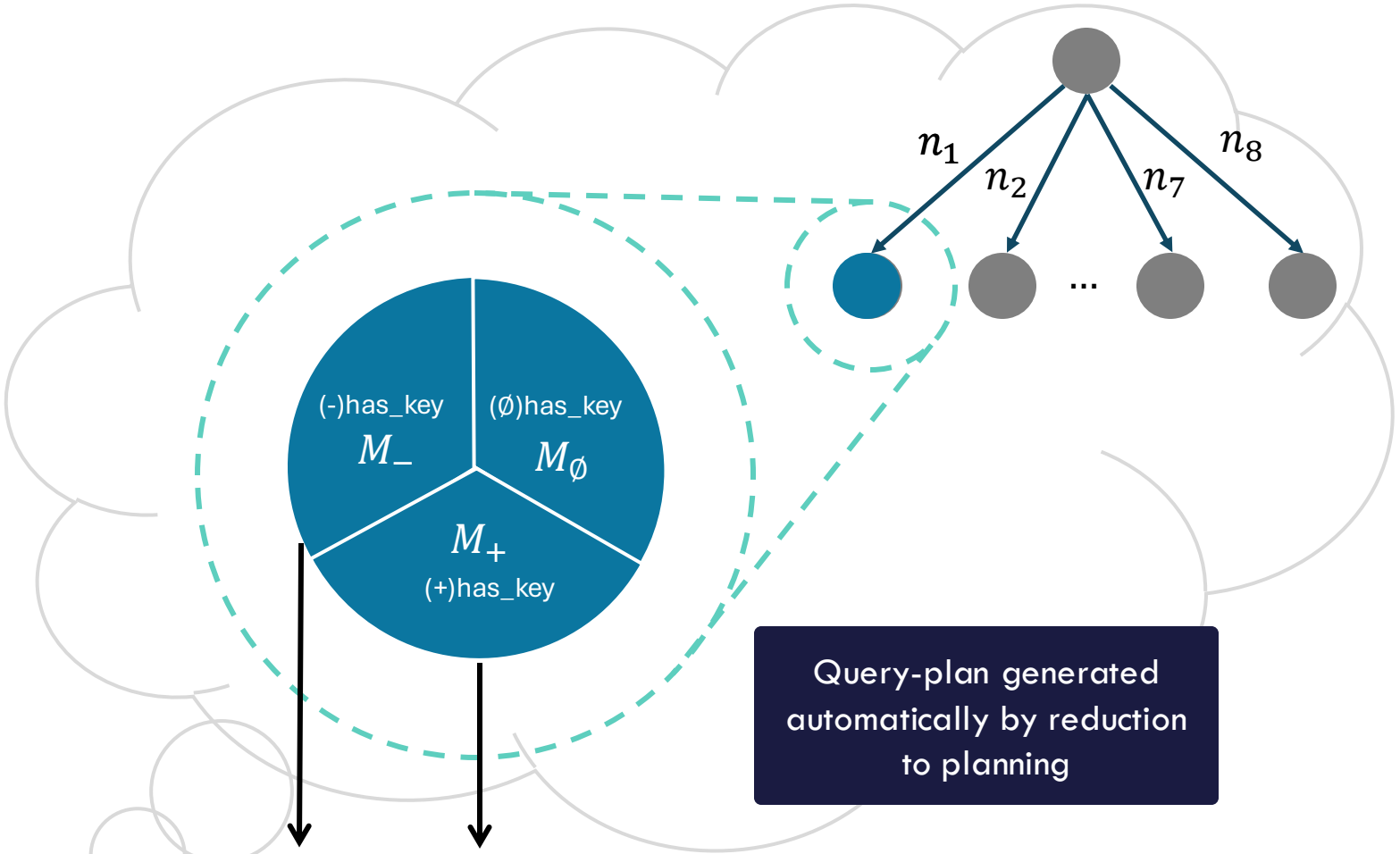
- Simple Query-Response Interface
- Should work for a variety of taskable AI systems.
  - Independent of their internals.
- Requirement: ⟨QueryType, ResponseType, ρ⟩.

Simple Queries

	Plan Outcome Queries	State Reachability Query
Query	In state s_I, what will happen if you execute the plan $\pi = \langle c_1, \ldots, c_n \rangle$?	Can you go from state s_I to state s_F?
Response	I can execute first ℓ steps of the plan, ending up in state s_F.	Yes / No.

- How to generate the queries?
- How to use the responses to generate models?

Hierarchical Query Synthesis

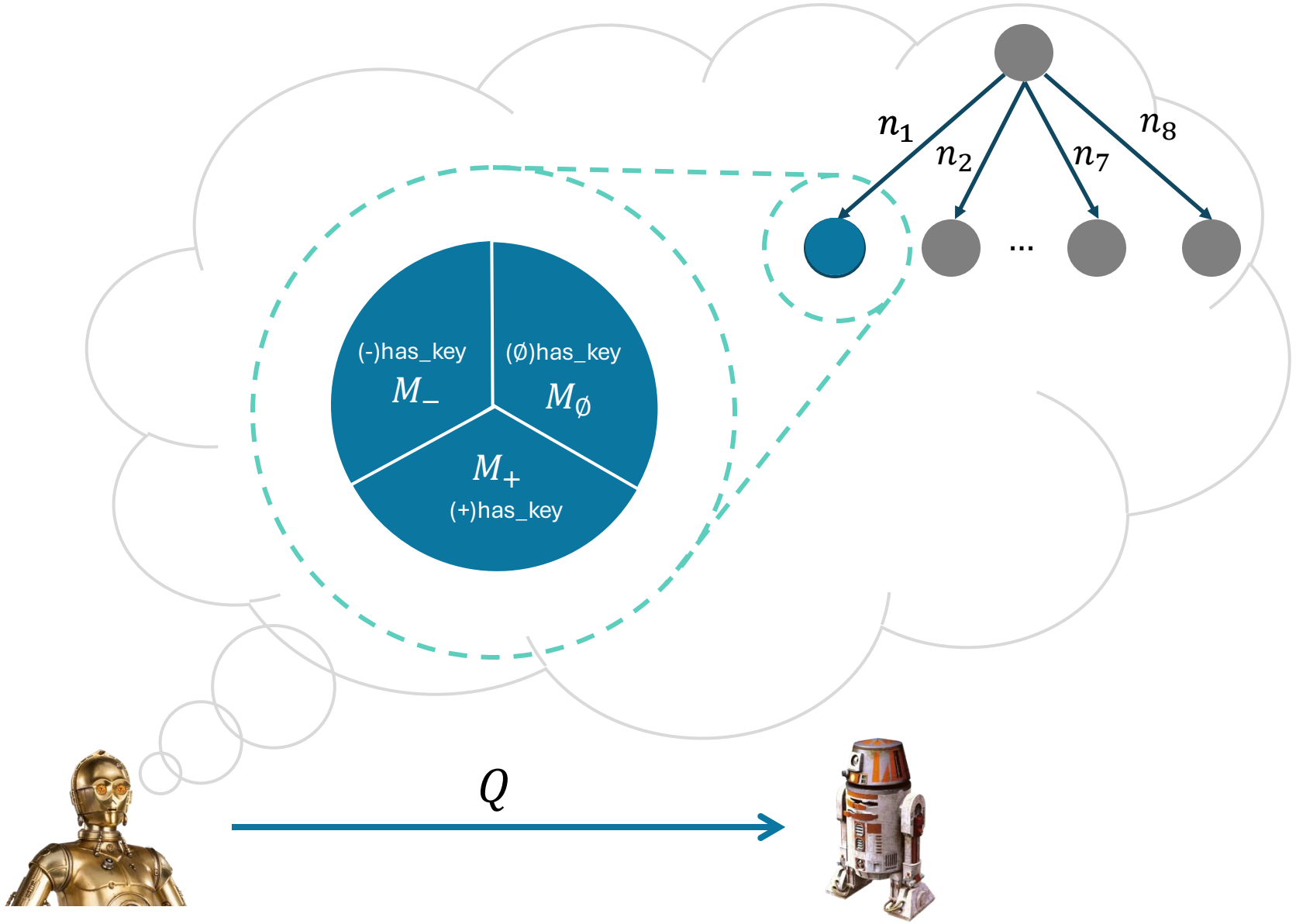


Query-plan generated automatically by reduction to planning

Generate a *distinguishing query:*
Q such that $Q(M_-) \neq Q(M_+)$

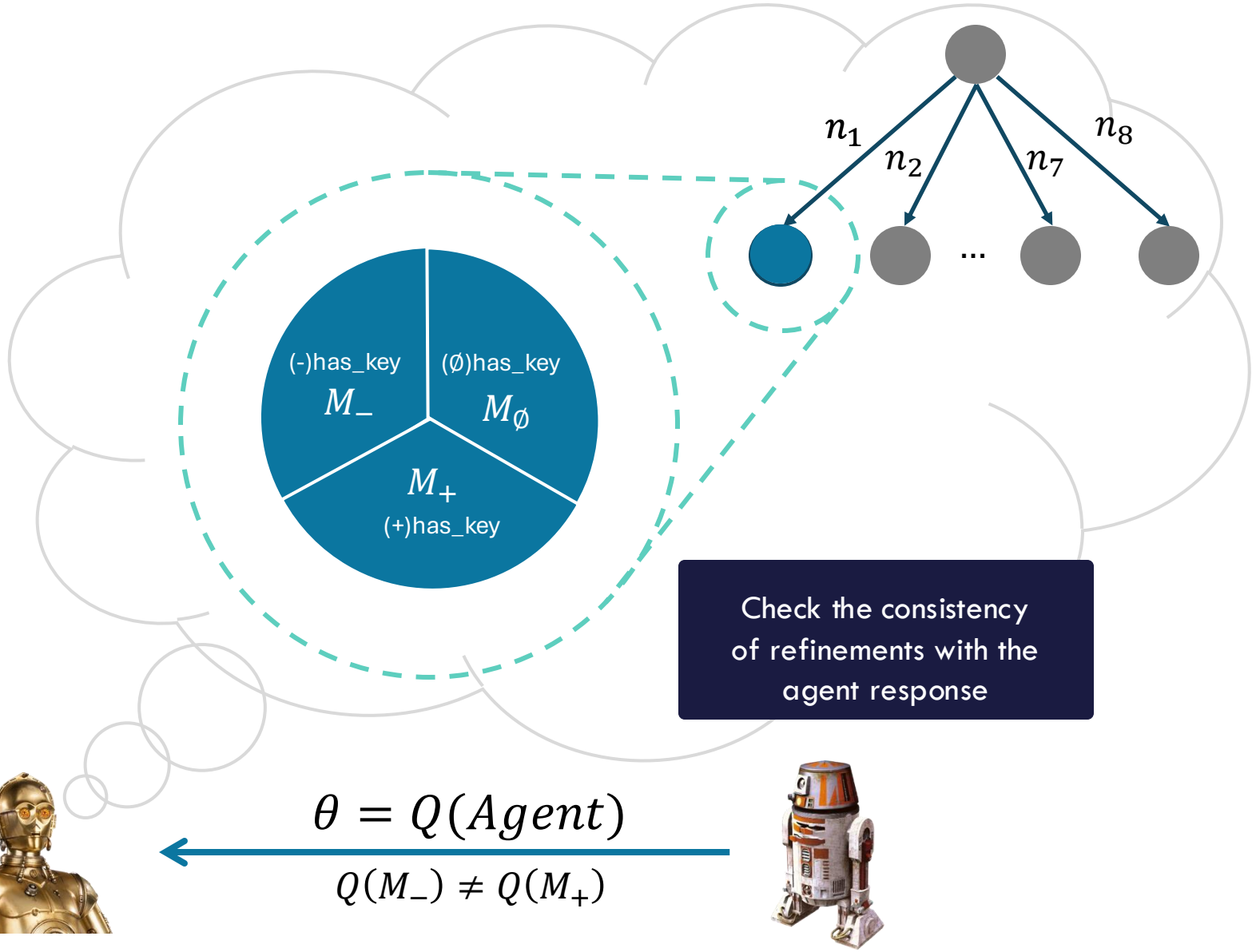
```
(:action open-door
  :parameters (?l1)
  :precondition (and
n1 (+/-/∅)(has_key)
n2 (+/-/∅)(door_open)
n3 (+/-/∅)(door_adjacent ?l1)
n4 (+/-/∅)(player_at ?l1))
  :effect (and
n5 (+/-/∅)(has_key)
n6 (+/-/∅)(door_open)
n7 (+/-/∅)(door_adjacent ?l1)
n8 (+/-/∅)(player_at ?l1))
```

Hierarchical Query Synthesis



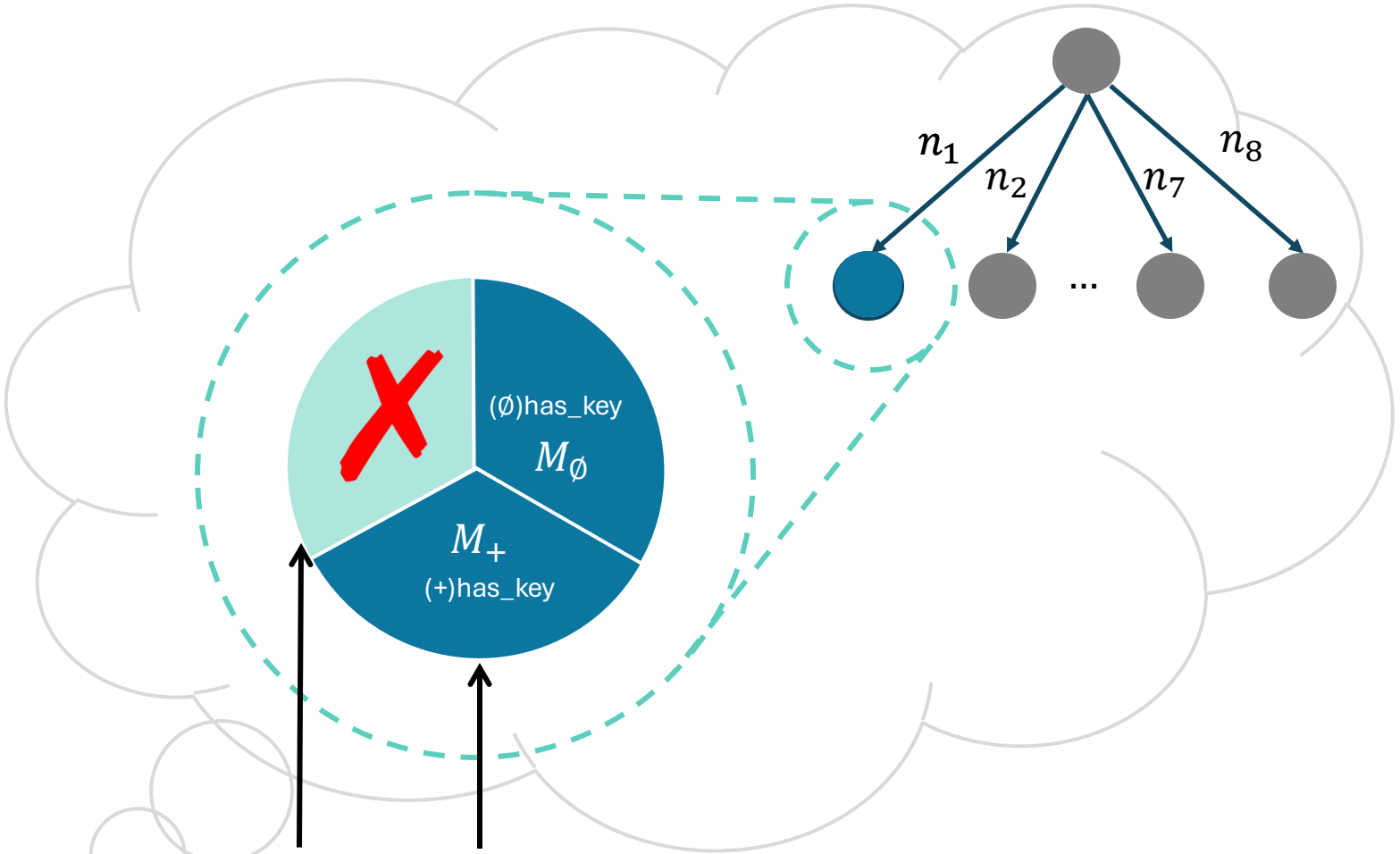

```
(:action open-door
  :parameters (?l1)
  :precondition (and
 n1  (+/-/∅)(has_key)
 n2  (+/-/∅)(door_open)
 n3  (+/-/∅)(door_adjacent ?l1)
 n4  (+/-/∅)(player_at ?l1))
  :effect (and
 n5  (+/-/∅)(has_key)
 n6  (+/-/∅)(door_open)
 n7  (+/-/∅)(door_adjacent ?l1)
 n8  (+/-/∅)(player_at ?l1))
```

Hierarchical Query Synthesis




```
(:action open-door
  :parameters (?l1)
  :precondition (and
n1  (+/-/∅)(has_key)
n2  (+/-/∅)(door_open)
n3  (+/-/∅)(door_adjacent ?l1)
n4  (+/-/∅)(player_at ?l1))
  :effect (and
n5  (+/-/∅)(has_key)
n6  (+/-/∅)(door_open)
n7  (+/-/∅)(door_adjacent ?l1)
n8  (+/-/∅)(player_at ?l1))
```

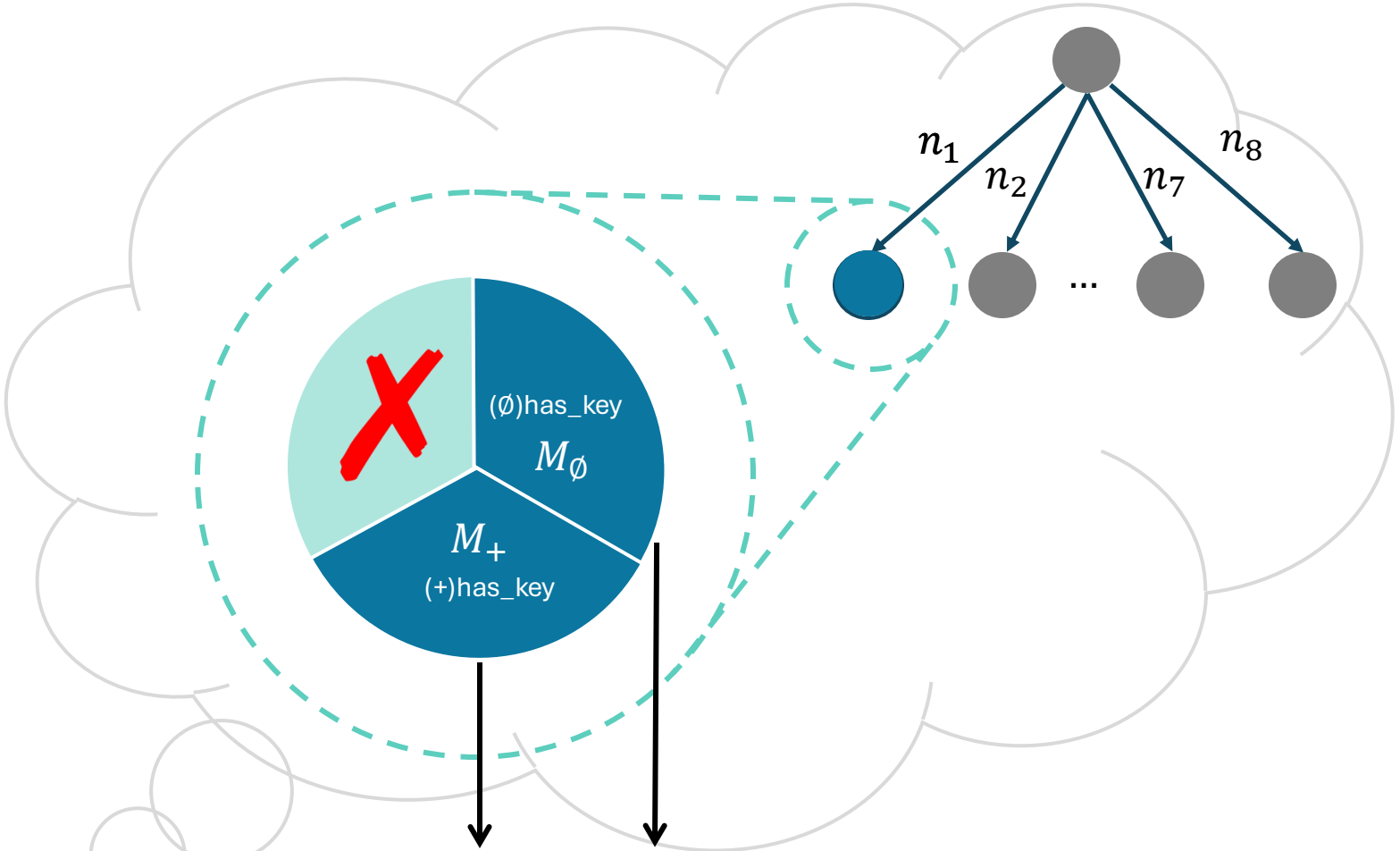
Hierarchical Query Synthesis




```
(:action open-door
  :parameters (?l1)
  :precondition (and
n1    (+/∅)(has_key)
n2   (+/-/∅)(door_open)
n3   (+/-/∅)(door_adjacent ?l1)
n4   (+/-/∅)(player_at ?l1))
  :effect (and
n5   (+/-/∅)(has_key)
n6   (+/-/∅)(door_open)
n7   (+/-/∅)(door_adjacent ?l1)
n8   (+/-/∅)(player_at ?l1))
```

Reject abstract model(s) that are not consistent with the agent

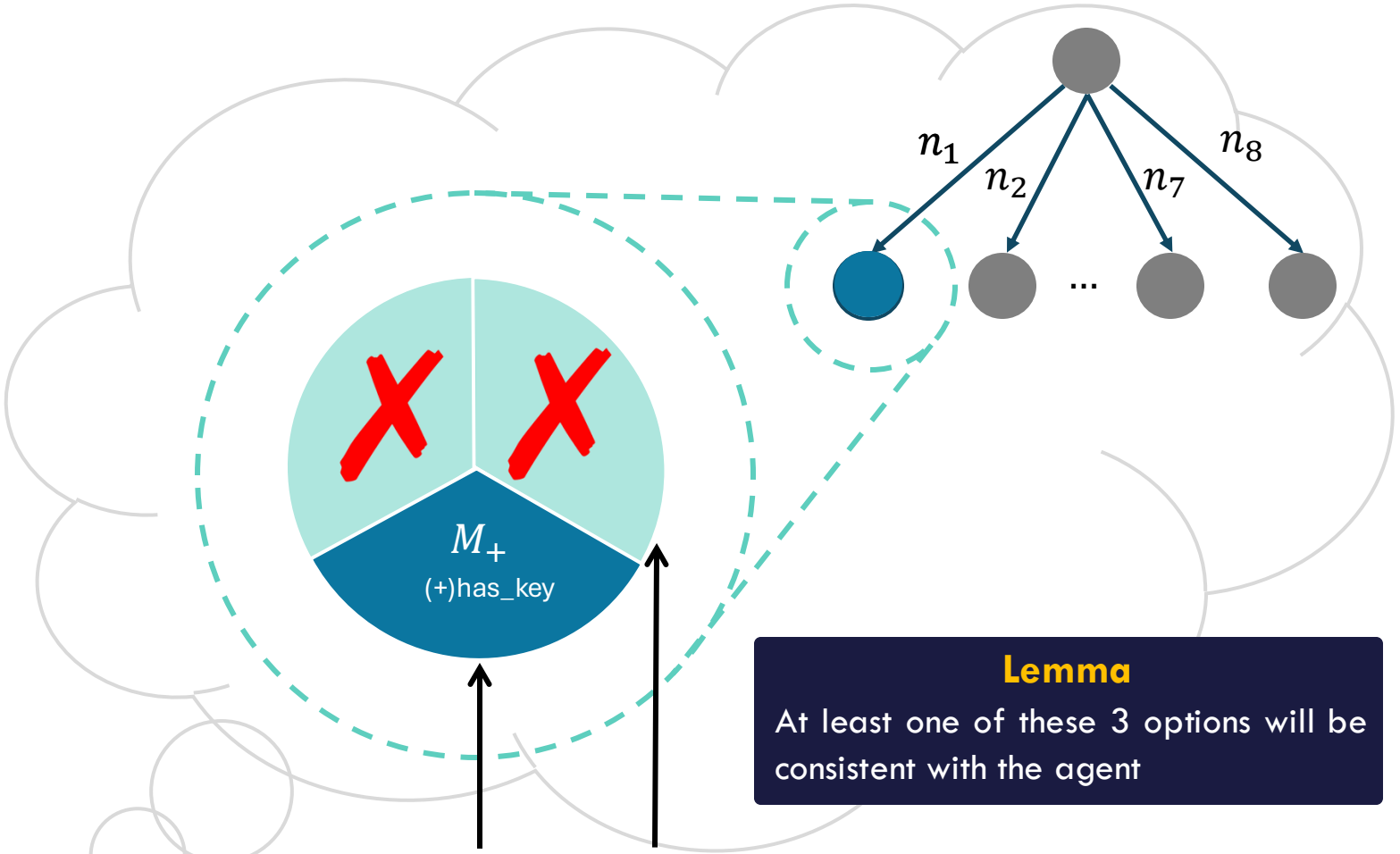
Hierarchical Query Synthesis




```
(:action open-door
  :parameters (?l1)
  :precondition (and
n1    (+/Ø)(has_key)
n2   (+/-/Ø)(door_open)
n3   (+/-/Ø)(door_adjacent ?l1)
n4   (+/-/Ø)(player_at ?l1))
  :effect (and
n5   (+/-/Ø)(has_key)
n6   (+/-/Ø)(door_open)
n7   (+/-/Ø)(door_adjacent ?l1)
n8   (+/-/Ø)(player_at ?l1))
```

Generate a distinguishing query
for these two abstract models

Hierarchical Query Synthesis



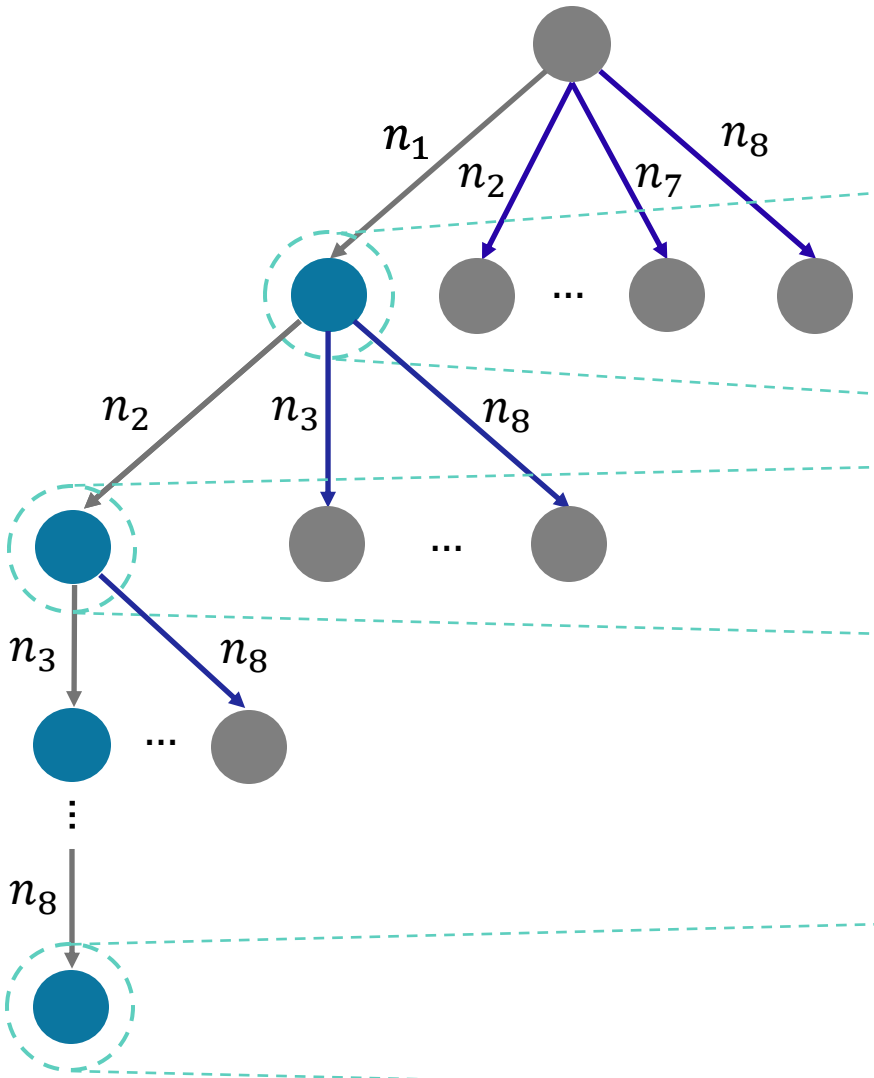
Lemma

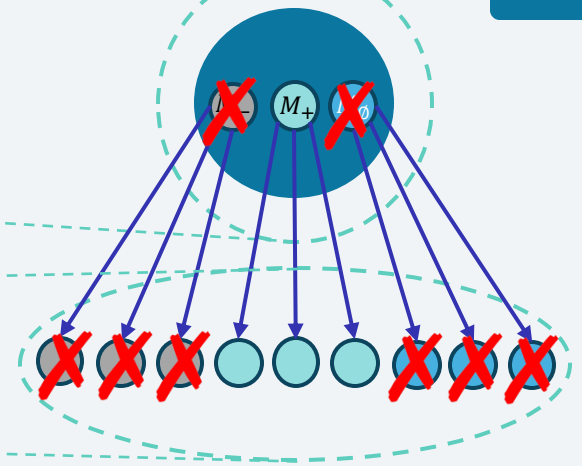
At least one of these 3 options will be consistent with the agent

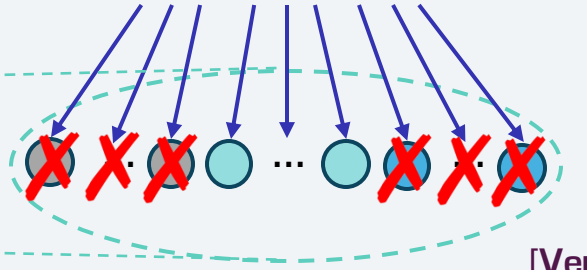
Reject the abstract model
that is not consistent with the agent

```
(:action open-door
  :parameters (?l1)
  :precondition (and
n1      (+)(has_key)
n2    (+/-/∅)(door_open)
n3    (+/-/∅)(door_adjacent ?l1)
n4    (+/-/∅)(player_at ?l1))
  :effect (and
n5    (+/-/∅)(has_key)
n6    (+/-/∅)(door_open)
n7    (+/-/∅)(door_adjacent ?l1)
n8    (+/-/∅)(player_at ?l1))
```

Hierarchical Query Synthesis







Key feature of the algorithm

Whenever we prune an abstract model, we prune a large number of concrete models.

Active Learning

[**Verma**, Marpally, Srivastava; AAAI '21]

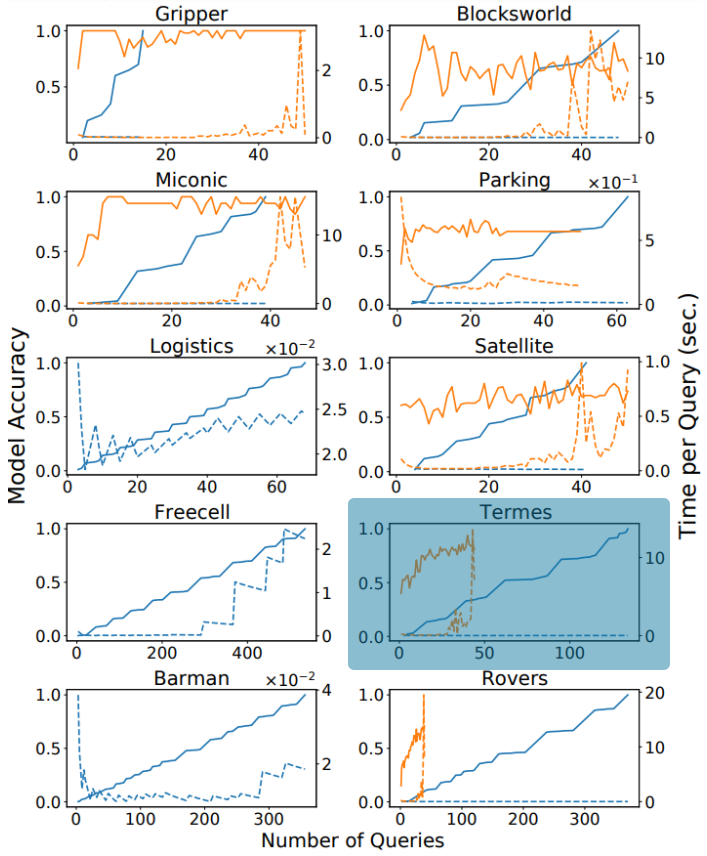
AAM learns Accurate Model with fewer Queries

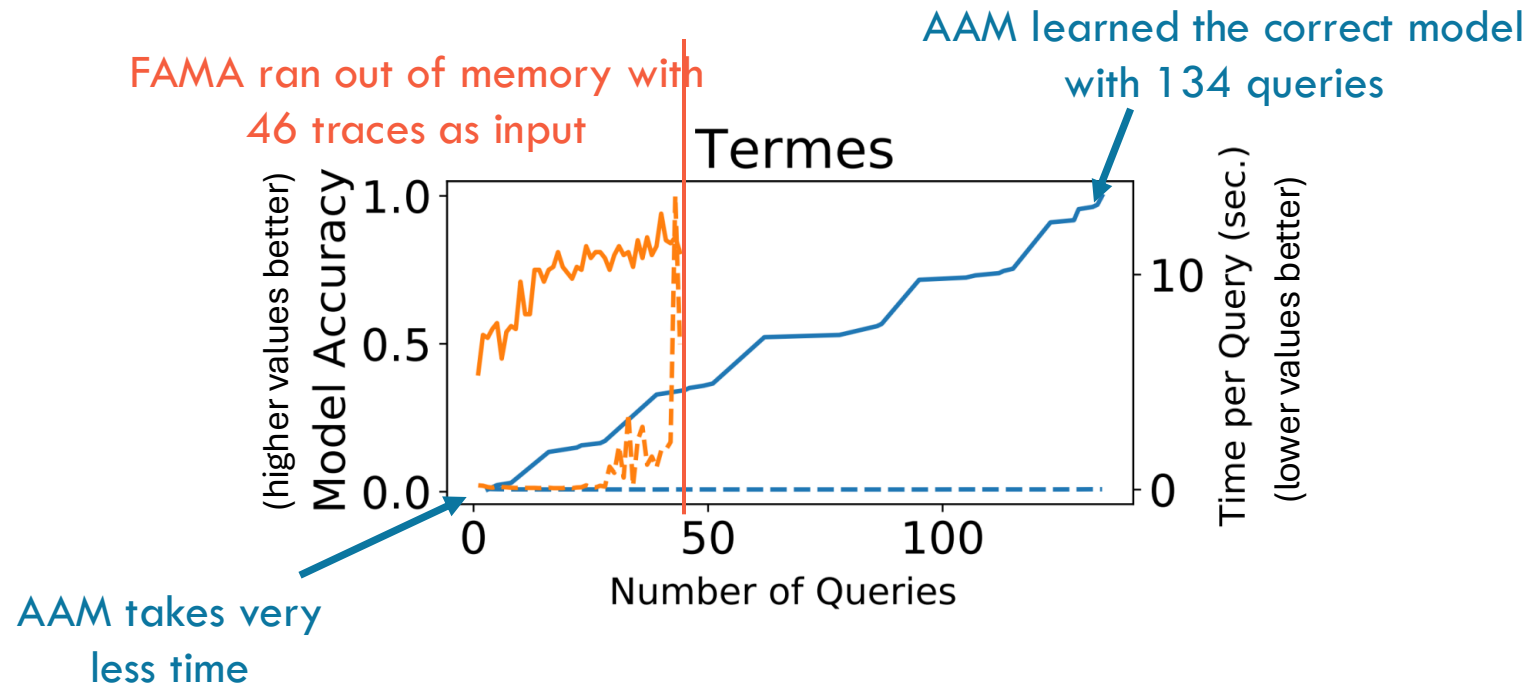
- Asses by learning the model and compare with ground truth.
- Baseline†: A passive learner (FAMA) that observes agent behavior

Random deterministic planning agent from IPC

Accuracy: AAM FAMA

Time: AAM FAMA





†Aineto, D.; Celorrio, S. J.; and Onaindia, E. 2019. *Learning Action Models With Minimal Observability*. Artificial Intelligence 275: 104–137.

AAM learns Accurate Deterministic Models

- Theorem (*termination*) : The algorithm terminates after a finite number of iterations.
- Theorem (*soundness*): The resulting (set of) model(s) is(are) functionally equivalent to the ground truth model.

Theorem 3 in Paper

Autonomous Capability Assessment of Sequential Decision-Making Systems in Stochastic Settings

Pulkit Verma, Rushang Karia, and Siddharth Srivastava

NeurIPS 2023

Stochastic and Stationary Setting

Input

- Predicates (User vocabulary)
  - With their evaluation functions
- List of capabilities.

Output

- PPDDL-like description of each capability.

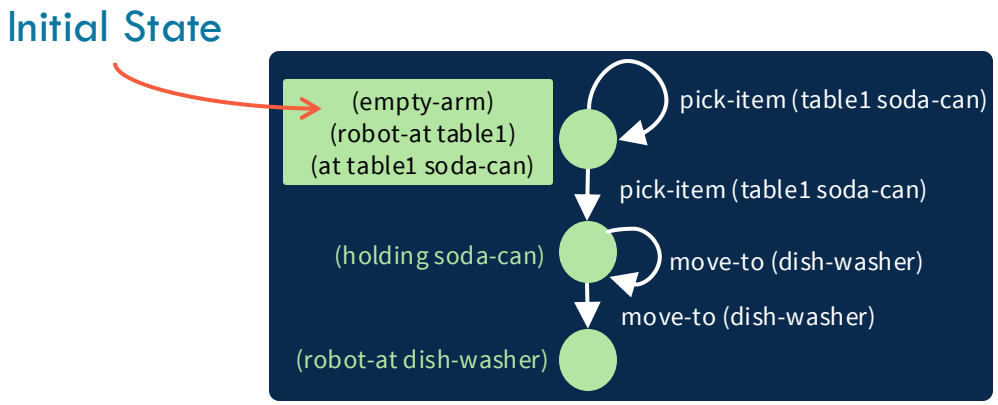
[**Verma**, Karia, Srivastava; NeurIPS '23]

Assumptions

- User's vocabulary matches simulator's vocabulary.
- Black-Box AI provides a list of capabilities.
- Stationary agent model.
- ~~Deterministic~~ Stochastic environment.
- Fully observable setting.

Changes for Stochastic Settings

New Queries



Policy: Generated Autonomously by Reduction to Non-Deterministic Planning

What happens if you start in the given initial state and follow this partial policy?

Assumptions

- User's vocabulary matches simulator's vocabulary.
- Black-Box AI provides a list of capabilities.
- Stationary agent model.
- ~~Deterministic~~ Stochastic environment.
- Fully observable setting.

Changes for Stochastic Settings

Step 1: Learn a Non-Deterministic Model

```
(:action open-door
  :parameters (?l1)
  :precondition (and
    (+/-/∅)(has_key)
    (+/-/∅)(door_open)
    (+/-/∅)(door_adjacent ?l1)
    (+/-/∅)(player_at ?l1))
  :effect (oneof
    (and
      (+/-/∅)(has_key)
      (+/-/∅)(door_open)
      (+/-/∅)(door_adjacent ?l1)
      (+/-/∅)(player_at ?l1))
    (and
      (+/-/∅)(has_key)
      (+/-/∅)(door_open)
      (+/-/∅)(door_adjacent ?l1)
      (+/-/∅)(player_at ?l1))))
```

Apply Maximum Likelihood Estimation on the observed data (query responses)

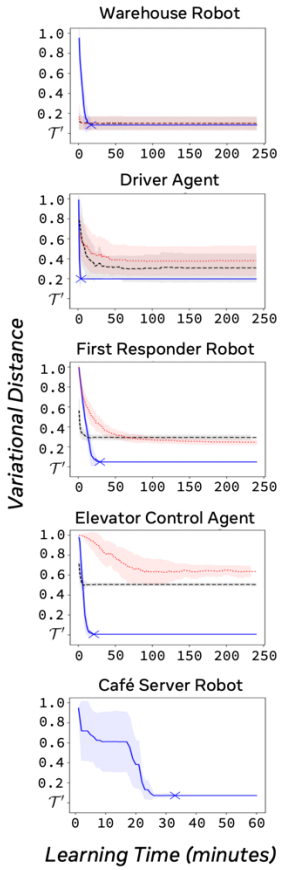
Step 2: Convert to Probabilistic Model

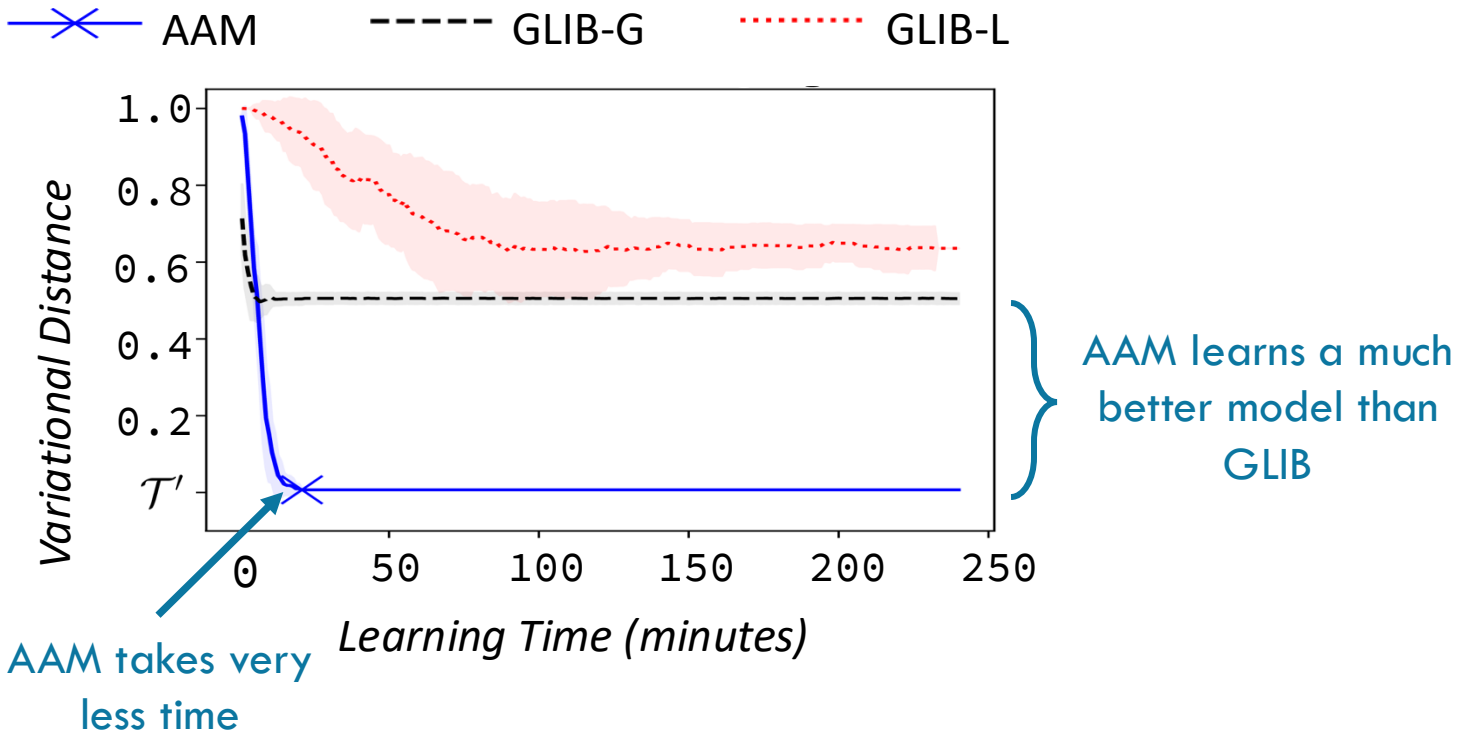
```
(:action open-door
  :parameters (?l1)
  :precondition (and
    (+/-/∅)(has_key)
    (+/-/∅)(door_open)
    (+/-/∅)(door_adjacent ?l1)
    (+/-/∅)(player_at ?l1))
  :effect (probabilistic
    0.xx (and
      (+/-/∅)(has_key)
      (+/-/∅)(door_open)
      (+/-/∅)(door_adjacent ?l1)
      (+/-/∅)(player_at ?l1))
    0.yy (and
      (+/-/∅)(has_key)
      (+/-/∅)(door_open)
      (+/-/∅)(door_adjacent ?l1)
      (+/-/∅)(player_at ?l1))))
```

AAM learns accurate probabilistic models faster

- Baseline: directed exploration approach (GLIB)
- Increase the time taken to learn the model.

Random probabilistic planning agent from IPC



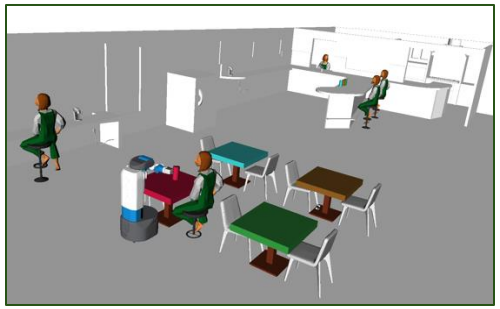


†Chitnis, R.; Silver, T.; Tenenbaum, J.; Kaelbling, L. P.; Lozano-Perez, T. GLIB: Efficient Exploration for Relational MBRL via Goal-Literal Babbling. AAAI 2021.

AAM learns accurate models for Continuous Domains

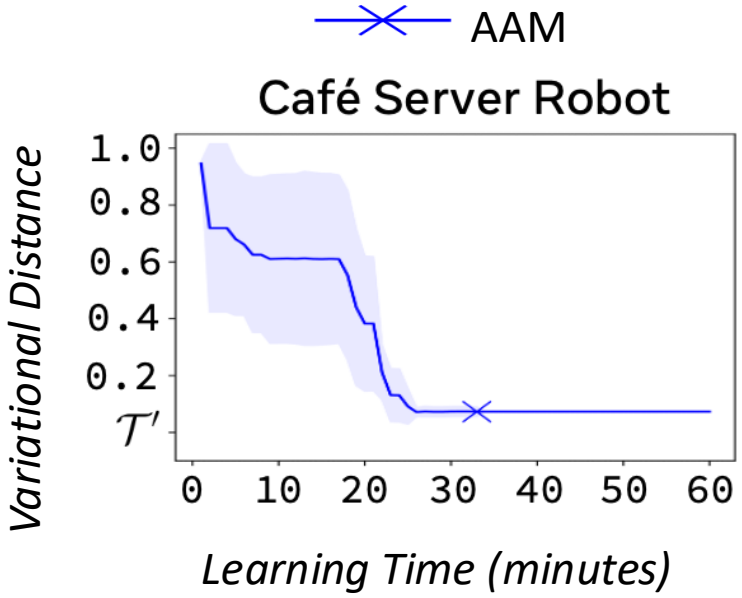
- Use Task and Motion Planning (TMP) to convert actions into motion plans.
- Increase the time taken to learn the model.

Probabilistic planning agent using OpenRave and TMP



	x	y	z	θ	φ	ψ
robot-base	1.0	-3.2	4.7	0.9	1.3	3.1
soda-can1	6.0	-2.8	3.5	8.3	6.7	9.2
⋮					⋮	
table4	-2.1	4.1	1.9	3.7	9.5	4.8

(empty-arm)
(robot-at table1)
(at table1 soda-can)



AAM learns Accurate Probabilistic Models

- Theorem (*soundness and completeness*):
  The intermediate non-deterministic model (after step 1) is sound and complete w.r.t. the ground truth model.

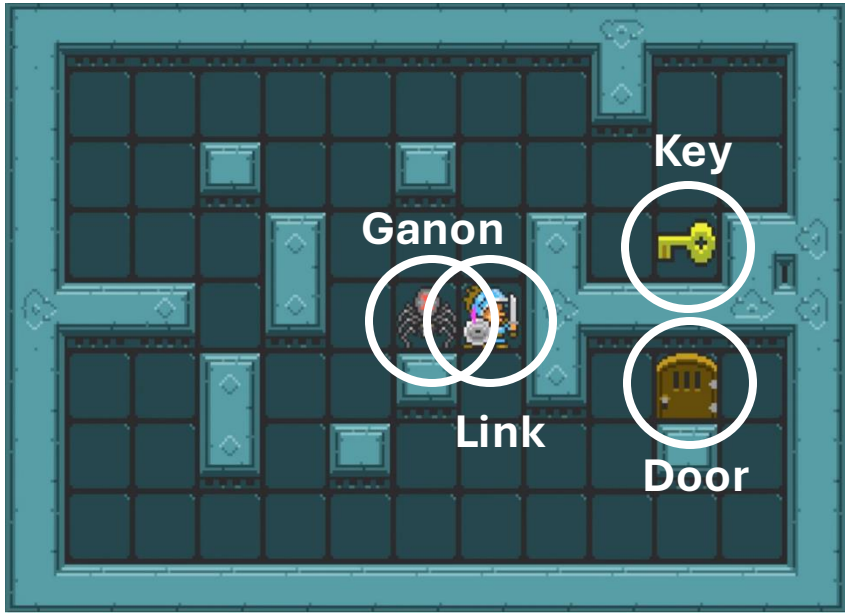
- Theorem (*probabilistic correctness*):
  The resulting probabilistic model is correct w.r.t. the ground truth model.

Discovering User-Interpretable Capabilities of Black-Box Planning Agents

Pulkit Verma, Shashank Rao Marpally, and Siddharth Srivastava

KR 2022

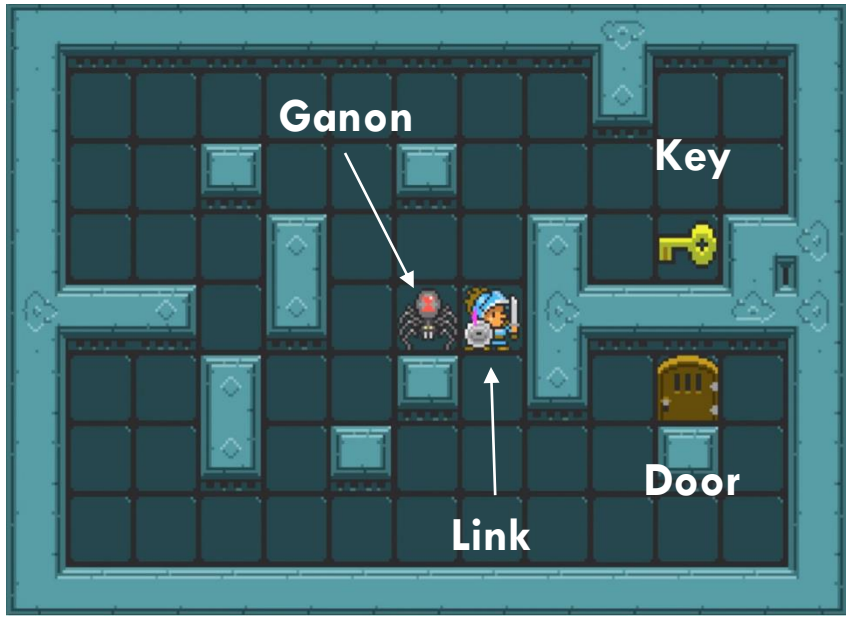
User Vocabulary can be Less Expressive



Agent's State Representation	State Representation in User's Vocabulary
pixel_1_1(#42A8B3)	(at ganon 5,3)
pixel_1_2(#42A8B3)	(at link 6,3)
.	(at key 9,4)
.	(at door 9,2)
.	
pixel_n_m(#203A3D)	

Capability v/s Functionality

- Functionality: Set of possible low-level actions of the agent.
- Capability: What agent's planning and learning algorithms can do.



Agent Actions (Keystrokes)	Learned Capabilities
W	(defeat ganon)
A	(go to door)
S	(go to key)
D	(go to ganon)
E	(pick key)
	(open door)

Knowledge of primitive actions might be insufficient to understand the agent's capabilities

Discovering Capabilities

Input

- Predicates (User vocabulary)
  - With their evaluation functions
- Samplers: high-level state to low-level state.
- Low-level state transitions.

Output

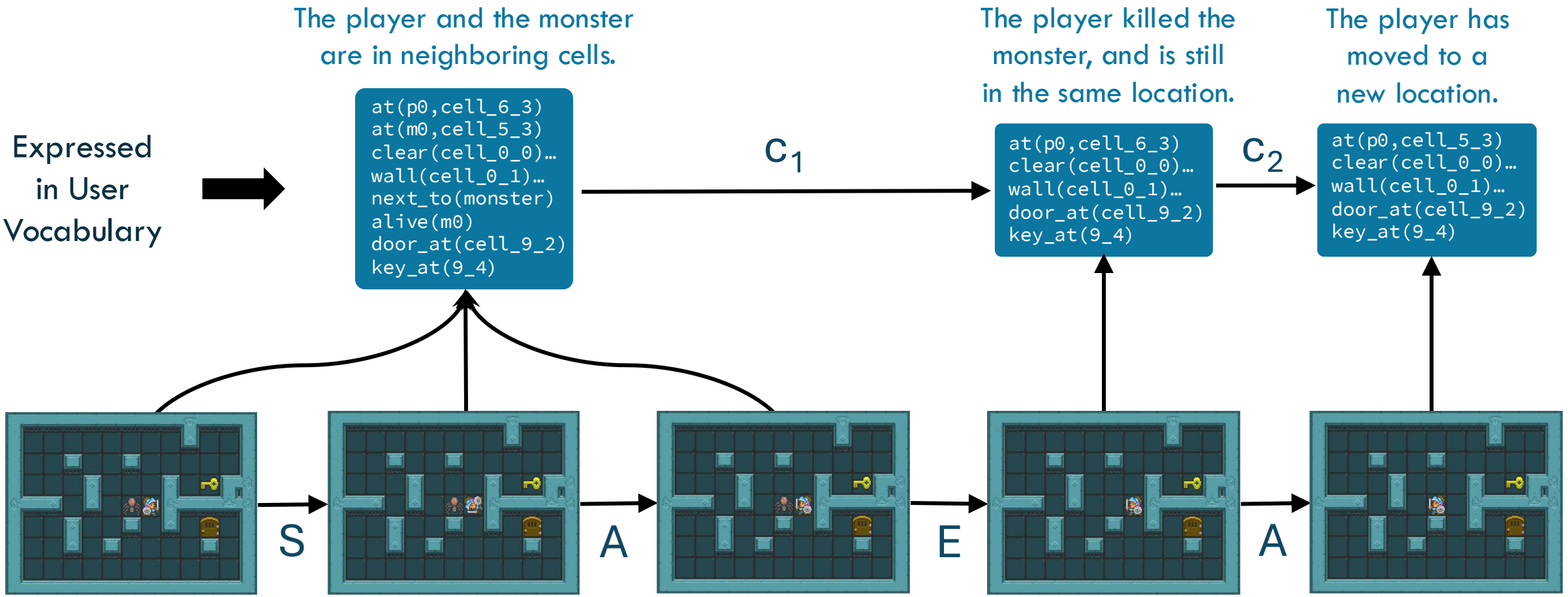
- List of capabilities.
- PDDL-like description of each capability.

[**Verma**, Marpally, Srivastava; KR '22]

Assumptions

- ~~User's vocabulary matches simulator's vocabulary.~~
- Black-Box AI provides a list of ~~capabilities.~~ transitions.
- Stationary agent model.
- Deterministic environment.
- Fully observable setting.

Discovering Capabilities using Input Predicates as Abstractions



Example of a Learned Capability Description

```
(:capability c4
 :parameters (?player1 ?cell1
   ?monster1 ?cell2)
 :precondition
  (and (alive ?monster1)
    (at ?player1 ?cell1)
    (at ?monster1 ?cell2)
    (next_to ?monster1))
 :effect
  (and (clear ?cell2)
   (not(alive ?monster1))
   (not(at ?monster1 ?cell2))
   (not(next_to ?monster1))))
```

Position of Link has not changed

Ganon is not at its previous location

Ganon is not alive anymore

Link is not next to Ganon

This capability is: “Defeat Ganon”

User Study Setup to Verify Interpretability

Preconditions

Effects

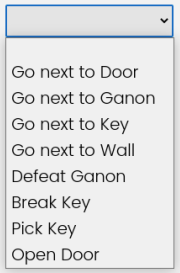
4. **Capability C4**:
The *player* can execute this capability when:

- The *monster* is not defeated.
- The *player* is in *cell1*.
- The monster is in *cell2*.
- The player is in a cell adjacent to the *monster*.

After the *player* executes this capability:

- *Cell2* is empty.
- The *monster* is defeated.
- The *monster* is not in *cell2*.
- The player is not in a cell adjacent to the *monster*.

Question 4 of 12:
Select the phrase that best summarizes the capability **C4**? We will use your response while referring to this capability **C4** later in the survey.



Possible options to choose from

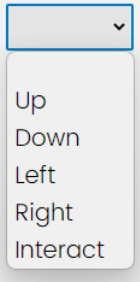
[Capability Description Example]

Keystroke Description

W: Pressing this key does the following:

- If Link is facing up and there is no wall, door, or key in the cell above, then Link moves to the cell above.
- If there is a wall, door, or key in the cell above Link, then Link stays in the same cell.
- If Link is facing Left, Right, or Down before pressing W, then Link faces up but stays in the same cell.

Question 1 of 11:
Select the phrase that best summarizes pressing **W**? We will use your response while referring to this key **W** later in the survey.



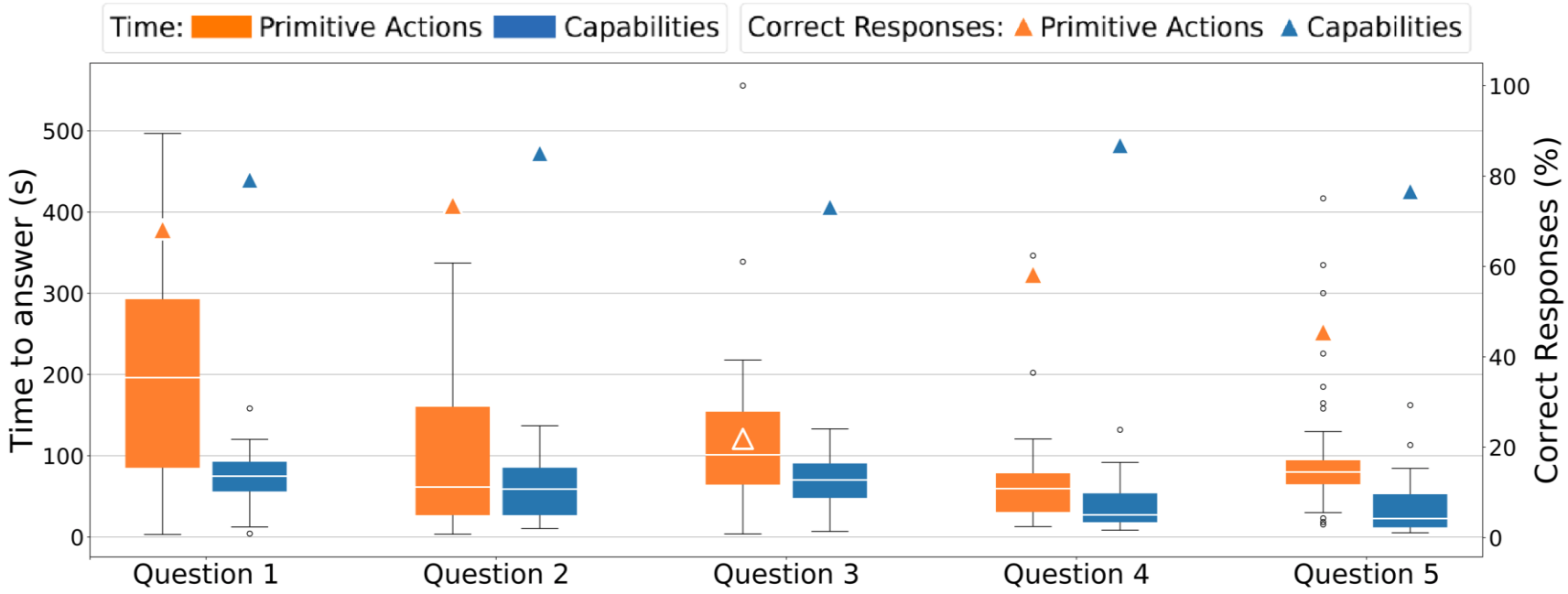
Possible options to choose from

[Functionality Description Example]

Utility of Discovered Capability Descriptions

If Link starts in the state shown below:

Which sequence of actions can Link take to reach the state shown below:



Learned Capability Descriptions are Maximally Consistent

- Theorem (*consistency*):
  The learned descriptions are consistent with the observations and the queries.

- Theorem (*maximal consistency*):
  This approach is maximally consistent, i.e., we cannot add any more literals to the preconditions or effects without ruling out some truly possible models.

- Theorem (*probabilistic completeness*):
  In the limit of infinite execution traces, the probability of discovering all capabilities expressible in the user vocabulary is 1.

Differential Assessment of Black-Box AI Agents

Rashmeet Kaur Nayyar, Pulkit Verma*, and Siddharth Srivastava*

AAAI 2022

Differential Assessment

Input

- Initial model of the AI system.
  - Predicates (User vocabulary)
    - With their evaluation functions
  - List of capabilities.
- Observations of AI system working in the environment.

Output

- Updated PDDL-like description of each capability.

Can we learn an updated model without doing a complete assessment?

Assumptions

- User's vocabulary matches simulator's vocabulary.
- Black-Box AI provides a list of capabilities.
- Adaptive ~~Stationary~~ agent model.
- Deterministic environment.
- Fully observable setting.

[Nayyar*, **Verma***, Srivastava; AAAI '22]

Agent updates

E.g., software update,

new deployment,
adapted for user needs, etc.

simulator

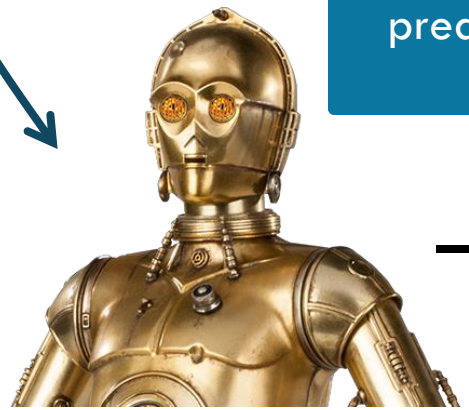
Observations
(collected once)

Query

Response

Use observations and M_{init} t
predict what might've
changed.

Initial Model
known to the user
M_{init}



Personalized
AI-Assessment Module

Updated model M_{drift} of

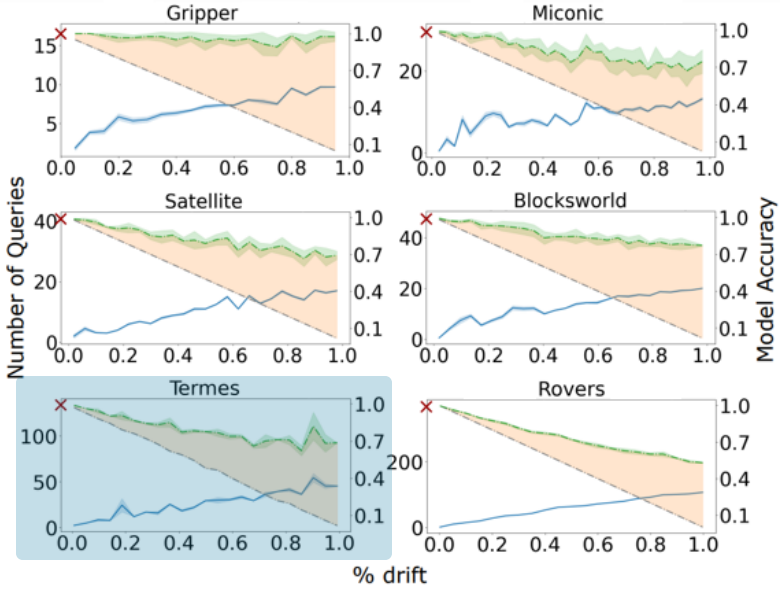
Black-Box AI System's
capabilities

[Nayyar*, **Verma***, Srivastava; AAAI '22]

Fewer Queries Needed Compared to Learning from Scratch

Random deterministic planning agent from IPC

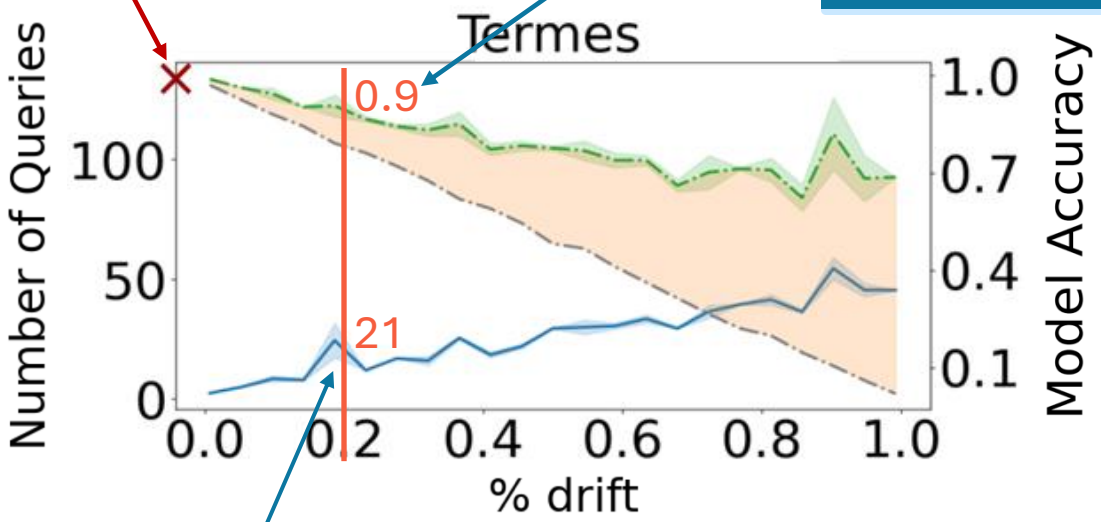
Accuracy of initial model
Accuracy gained by AAM
Accuracy of model computed by AAM
Number of queries by AAM
× Number of queries when learning from scratch



134 queries needed if starting from scratch

Accuracy

Used 10 observations per domain



Number of queries are much lower than 134

Learned Updated Capability Descriptions are Consistent

- Theorem (*consistency*): The learned descriptions are consistent with the observations and the query responses.

Interpretability Analysis of Symbolic Representations for Sequential Decision-Making Systems

Pulkit Verma and Julie A. Shah

HRI 2025 Workshop on Explainability for Human-Robot Collaboration

Interpretable Representations

Representations beyond PDDL.

- Temporal Logic (LTL/STL)
- Bayesian Networks
- RDDL

How interpretable each representation is?

Which representation fits the requirements of end user well?

Classification Framework

Dimension	Description
Formalism Type	Symbolic, Subsymbolic, or Hybrid
Interpretability Level	How easily humans understand the representation
Temporal Expressiveness	How well time-related concepts are represented
Abstraction Level	Level of detail in the representation
Explanation Type	How explanations are generated
Domain Specificity	General vs. domain-specific applicability
Human Interaction	How humans interface with the representation

P. Verma, J. A. Shah. Interpretability Analysis of Symbolic Representations for Sequential Decision-Making Systems. HRI 2025 Workshop on Explainability for Human-Robot Collaboration, 2025

Classification Framework

Representation	Interpretability Level	Temporal Expressiveness	Abstraction Level	Explanation Type	Domain Specificity	Human Interaction
Markov Decision Processes (MDPs)	Medium	Discrete Time	Low-Level	Global	General	Indirect
Finite State Machines (FSMs)	Medium	Discrete Time	Low-Level	Global	General	Direct
Decision Trees	High	N/A	Low-Level	Global	General	Direct
Rule-Based Systems	High	N/A	Low-Level	Global	General	Direct
Temporal Logic (LTL, STL, etc.)	Medium	Continuous Time	Low-Level	Global	Domain	Indirect
Program Synthesis	Low	N/A	High-Level	Global	Domain	Indirect
Planning Domain Definition Language (PDDL)	Medium	Discrete Time	High-Level	Global	Domain	Indirect
Causal Models	Medium	N/A	Multi-Level	Global	General	Indirect
Neuro-Symbolic Integration	Low	N/A	Multi-Level	Global	General	Indirect

P. Verma, J. A. Shah. Interpretability Analysis of Symbolic Representations for Sequential Decision-Making Systems. HRI 2025 Workshop on Explainability for Human-Robot Collaboration, 2025

Rushang Karia

Shashank Marpally

Rashmeet Nayyar

Siddharth Srivastava

Julie A. Shah

bit.ly/icad25-workshop

Questions?

Interpretable Model Learning for Taskable AI Systems

Pulkit Verma
pulkitv@mit.edu
pulkitverma.net